

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)

Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа № 1.1
по дисциплине: Дискретная математика
по теме: Операции над множествами

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: ст. преподаватель
Бондаренко Т.В.

Белгород 2023

Цель работы: изучить и научиться использовать алгебру подмножеств, изучить различные способы представления множеств в памяти ЭВМ, научиться программно реализовывать операции над множествами и выражения в алгебре подмножеств.

Задания:

1. Вычислить значение выражения (см. “Варианты заданий”, п. а).

Во всех вариантах считать $U = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$. Решение изобразить с помощью кругов Эйлера.

2. Записать выражение в алгебре подмножеств, значение которого при заданных множествах А, В и С равно множеству D (см. “Варианты заданий”, п. б).

3. Программно реализовать операции над множествами, используя следующие способы представления множества в памяти ЭВМ:

а) элементы множества А хранятся в массиве А. Элементы массива А неупорядочены;

б) элементы множества А хранятся в массиве А. Элементы массива А упорядочены по возрастанию;

в) элементы множества А хранятся в массиве А, элементы которого типа boolean. Если $i \in A$, то $A_i = \text{true}$, иначе $A_i = \text{false}$.

4. Написать программы для вычисления значений выражений (см. “Задания”, п.1 и п.2).

5. Используя программы (см. “Задания”, п.4), вычислить значения выражений (см. “Задания”, п.1 и п.2).

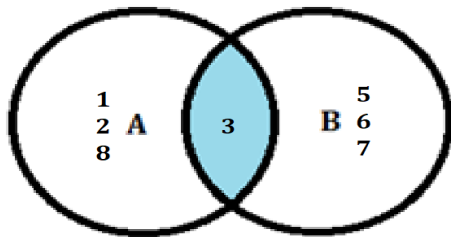
Задание 1 Вычислить значение выражения (см. “Варианты заданий”, п. а).

Во всех вариантах считать $U = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$. Решение

изобразить с помощью кругов Эйлера.

$$D = (A \cap B) \cup (A - C) \cup (B - C)$$

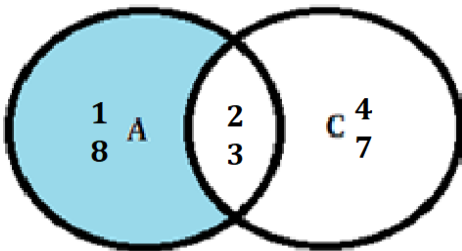
1) $(A \cap B)$



Решение:

$$\{1, 2, 3, 8\} \cap \{3, 5, 6, 7\} = \{3\}$$

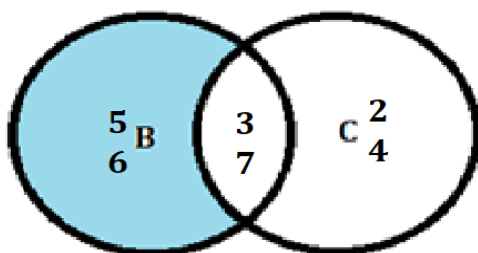
2) $(A - C)$



Решение:

$$\{1, 2, 3, 8\} - \{2, 3, 4, 7\} = \{1, 8\}$$

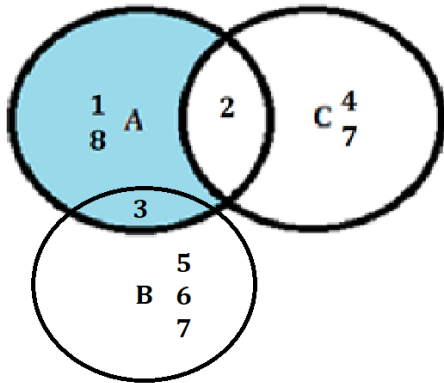
3 $(B - C)$



Решение:

$$\{3,5,6,7\} - \{2,3,4,7\} = \{5, 6\}$$

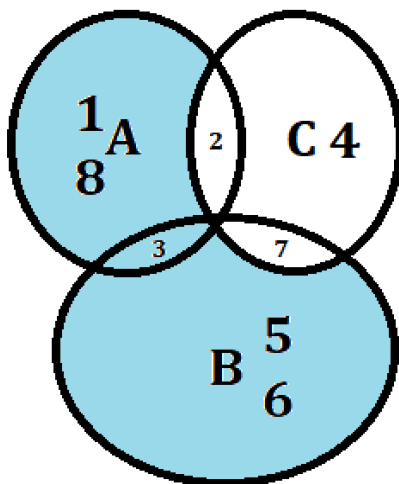
$$4) (A \cap B) \cup (A - C)$$



Решение:

$$(\{1,2,3,8\} \cap \{3,5,6,7\}) \cup (\{1,2,3,8\} - \{2,3,4,7\}) = \{1, 3, 8\}$$

$$5) (A \cap B) \cup (A - C) \cup (B - C)$$



Решение:

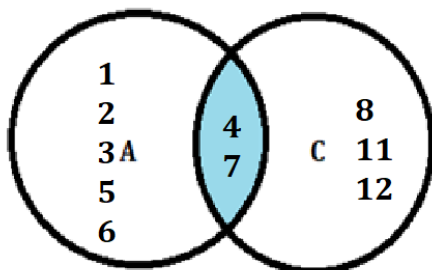
$$(\{1,2,3,8\} \cap \{3,5,6,7\}) \cup (\{1,2,3,8\} - \{2,3,4,7\}) \cup (\{3,5,6,7\} - \{2,3,4,7\}) = \{5, 6, 1, 3, 8\}$$

Задание 2 Записать выражение в алгебре подмножеств, значение которого при заданных множествах A, B и C равно множеству D

$A=\{1,2,3,4,5,6,7\}$ $B=\{2,5,6,9,10\}$ $C=\{4,7,8,11,12\}$

$D = \{2,5,6,7,4\}$

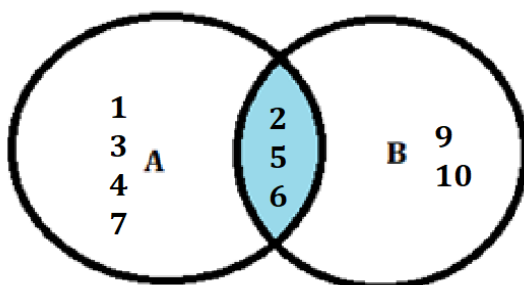
1) $(A \cap C)$



Ответ:

$\{4, 7\}$

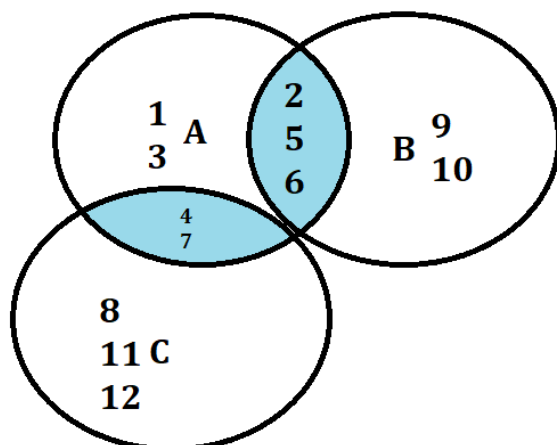
2) $(A \cap B)$



Ответ:

$\{2, 5, 6\}$

3) $(A \cap C) \cup (A \cap B)$



Ответ:

$\{2, 5, 6, 4, 7\}$

$D = \{2, 5, 6, 7, 4\} = \{2, 5, 6, 4, 7\} \Rightarrow$ Ответ на задание: $(A \cap C) \cup (A \cap B)$

Задание 3 Программно реализовать операции над множествами, используя следующие способы представления множества в памяти ЭВМ:

А) элементы множества А хранятся в массиве А. Элементы массива А неупорядочены

1) Объединение

```
#include <stdio.h>

void inputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
}

void outputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
}

void unionSets(int const *a, int sizeA, int const *b, int sizeB, int *c, int *sizeC) {
    int n = 0;
    //записываем все числа из массива а в массив с
    for (int i = 0; i < sizeA; i++) {
        c[n] = a[i];
        n++;
    }

    //если элемент из массива b не совпадает с элементом из массива а, к
    //счётчику добавляем 1
    int count = 0;
    for (int j = 0; j < sizeB; j++) {
        for (int k = 0; k < sizeA; k++) {
            if (b[j] != a[k]) {
                count++;
            }
        }

        //если счётчику стал равен размеру массива а (то есть число из массива
        //b не совпало с элементами массива а)
        //записываем этот элемент в массив с
        if (count == sizeA) {
            c[n] = b[j];
            n++;
        }

        count = 0;
    }

    //записываем размер массива с
    *sizeC = n;
}

int main() {
    int sizeA = 5;
```

```
int sizeB = 5;

int a[sizeA];
int b[sizeB];
int c[sizeA + sizeB];

inputArray(a, sizeA);

inputArray(b, sizeB);

int sizeC = 0;

unionSets(a, sizeA, b, sizeB, c, &sizeC);

outputArray(c, sizeC);

return 0;
}
```

2) Пересечение

```
#include <stdio.h>

void inputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
}

void outputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
}

void intersectionMany(int const * a, int sizeA, int const * b, int sizeB, int
*c, int *sizeC) {
    int n = 0;

    //находим элементы в массиве a и b, которые совпадают
    for (int i = 0; i < sizeA; i++) {
        for (int j = 0; j < sizeB; j++) {
            if (a[i] == b[j]) {
                //записываем эти элементы в массив c
                c[n++] = a[i];
            }
        }
    }

    //сохраняем размер массива c
    *sizeC = n;
}

int main() {
    int sizeA = 5;
    int sizeB = 3;

    int a[sizeA];
    int b[sizeB];
    int c[sizeA + sizeB];

    inputArray(a, sizeA);
    inputArray(b, sizeB);

    int sizeC = 0;
    intersectionMany(a, sizeA, b, sizeB, c, &sizeC);

    outputArray(c, sizeC);

    return 0;
}
```

3) Разность

```
#include <stdio.h>

void inputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
}

void outputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
}

void differenceSet(int const * a, int sizeA, int const * b, int sizeB, int
*c, int *sizeC) {
    int n = 0;

    int count = 0;
    for (int i = 0; i < sizeA; i++) {
        for (int j = 0; j < sizeB; j++) {
            if (a[i] != b[j]) {
                count++;
            }
        }
        if (count == sizeB) {
            c[n++] = a[i];
        }

        count = 0;
    }

    *sizeC = n;
}

int main() {
    int sizeA = 5;
    int sizeB = 3;

    int a[sizeA];
    int b[sizeB];
    int c[sizeA + sizeB];

    inputArray(a, sizeA);
    inputArray(b, sizeB);

    int sizeC = 0;
    differenceSet(a, sizeA, b, sizeB, c, &sizeC);

    outputArray(c, sizeC);

    return 0;
}
```

4) Симметрическая разность

```
#include <stdio.h>

void inputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
}

void outputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
}

void symmetricDifferenceSet(int const *a, int sizeA, int const *b, int sizeB,
int *c, int *sizeC) {
    int count = 0;

    //если элемент из массива a не совпадает с элементом из массива b к
счётчик добавляем 1
    int n = 0;
    for (int i = 0; i < sizeA; i++) {
        for (int z = 0; z < sizeB; z++) {
            if (a[i] != b[z]) {
                n++;
            }
        }

        //если счётчик равен размеру массива b (то есть элемента из массива a
нет в массиве b)
        //записываем элемент из массива a в массив c
        if (n == sizeB) {
            c[count++] = a[i];
        }

        n = 0;
    }

    //если элемент из массива b не совпадает с элементом из массива a к
счётчик добавляем 1
    int check = 0;
    for (int j = 0; j < sizeB; j++) {
        for (int k = 0; k < sizeA; k++) {
            if (b[j] != a[k]) {
                check++;
            }
        }

        //если счётчик равен размеру массива a (то есть элемента из массива b
нет в массиве a)
        //записываем элемент из массива b в массив c
        if (check == sizeA) {
            c[count++] = b[j];
        }

        check = 0;
    }

    //записываем размер массива c
    *sizeC = count;
}
```

```
int main() {  
    int sizeA = 5;  
    int sizeB = 3;  
  
    int a[sizeA];  
    int b[sizeB];  
    int c[sizeA + sizeB];  
  
    inputArray(a, sizeA);  
    inputArray(b, sizeB);  
  
    int sizeC = 0;  
    symmetricDifferenceSet(a, sizeA, b, sizeB, c, &sizeC);  
  
    outputArray(c, sizeC);  
  
    return 0;  
}
```

5) Включено ли множество в другое

```
#include <stdio.h>

void inputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
}

int checkingArray(int const * a, int sizeA, int const * b, int sizeB) {
    //если элемент из массива b равен элементу из массива a увеличиваем
    //счётчик на 1
    int count = 0;

    for (int i = 0; i < sizeB; i++) {
        for (int j = 0; j < sizeA; j++) {
            if (b[i] == a[j]) {
                count++;
                break;
            }
        }
    }

    //если счётчик равен размеру массива a (то есть все элементы из массива b
    //есть в массиве a)
    //возвращаем 1, иначе 0
    return count == sizeA ? 1 : 0;
}

int main() {
    int sizeA = 3;
    int sizeB = 5;

    int a[sizeA];
    int b[sizeB];

    inputArray(a, sizeA);
    inputArray(b, sizeB);

    if (checkingArray(a, sizeA, b, sizeB)){
        printf("Yes");
    } else {
        printf("No");
    }

    return 0;
}
```

6) Равны ли множества

```
#include <stdio.h>

void inputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
}

int checkingArray(int const * a, int sizeA, int const * b, int sizeB) {
    //проверяем равны ли длины массива, чтобы не делать лишних проверок
    if (sizeA == sizeB) {
        //определяем совпадают ли элемент элемент из массива a и массива b,
        //если не совпадают к счётчику добавляем 1
        int count = 0;

        for (int i = 0; i < sizeA; i++) {
            for (int j = 0; j < sizeB; j++) {
                if (a[i] != b[j]) {
                    count++;
                }
            }
        }

        //если счётчику равен размеру массива b(то есть был несовпадающий элемент),
        //сразу возвращаем 0
        if (count == sizeB) {
            return 0;
        }

        count = 0;

        //если все элементы оказались одинаковые возвращаем 1
        return 1;
    } else {
        return 0;
    }
}

int main() {
    int sizeA = 5;
    int sizeB = 5;

    int a[sizeA];
    int b[sizeB];

    inputArray(a, sizeA);
    inputArray(b, sizeB);

    if (checkingArray(a, sizeA, b, sizeB)){
        printf("Yes");
    } else {
        printf("No");
    }

    return 0;
}
```

7) Строгое включение

```
#include <stdio.h>

void inputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
}

int checkingArray(int const * a, int sizeA, int const * b, int sizeB) {
    //если элемент из массива b равен элементу из массива a увеличиваем
    //счётчик на 1
    int count = 0;

    for (int i = 0; i < sizeB; i++) {
        for (int j = 0; j < sizeA; j++) {
            if (b[i] == a[j]) {
                count++;
                break;
            }
        }
    }

    //если счётчик равен размеру массива a (то есть все элементы из массива b
    //есть в массиве a)
    //возвращаем 1, иначе 0
    return count == sizeA && sizeA != sizeB ? 1 : 0;
}

int main() {
    int sizeA = 3;
    int sizeB = 5;

    int a[sizeA];
    int b[sizeB];

    inputArray(a, sizeA);
    inputArray(b, sizeB);

    if (checkingArray(a, sizeA, b, sizeB)){
        printf("Yes");
    } else {
        printf("No");
    }

    return 0;
}
```

б) элементы множества А хранятся в массиве А. Элементы массива А упорядочены по возрастанию

1) Объединение

```
#include <stdio.h>

void inputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
}

void outputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
}

void unionSets(int const *a, int sizeA, int const *b, int sizeB, int *c, int *sizeC) {
    int n = 0;
    int i = 0;
    int j = 0;

    //пока индексы меньше размеров массива, ведём запись
    while (i < sizeA && j < sizeB) {
        //если элемент из массива а меньше элемента из массива b, записываем
        //его в массив c
        //иначе если элемент из массива b меньше элемента из массива a,
        //записываем его в массив c
        //иначе идём дальше, ведь элементы совпадают
        if (a[i] < b[j]) {
            c[n++] = a[i];
            i++;
        } else if (a[i] > b[j]) {
            c[n++] = b[j];
            j++;
        } else {
            i++;
        }
    }

    //дописываем в конец оставшие элементы из массива а пока индекс меньше
    //размера массива, если такие остались
    while (i < sizeA) {
        c[n++] = a[i];
        i++;
    }

    //дописываем в конец оставшие элементы из массива b пока индекс меньше
    //размера массива, если такие остались
    while (j < sizeB) {
        c[n++] = b[j];
        j++;
    }

    //записываем размер массива c
    *sizeC = n;
}
```

```
int main() {  
    int sizeA = 5;  
    int sizeB = 5;  
  
    int a[sizeA];  
    int b[sizeB];  
    int c[sizeA + sizeB];  
  
    inputArray(a, sizeA);  
    inputArray(b, sizeB);  
  
    int sizeC = 0;  
    unionSets(a, sizeA, b, sizeB, c, &sizeC);  
  
    outputArray(c, sizeC);  
  
    return 0;  
}
```

2) Пересечение

```
#include <stdio.h>

void inputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
}

void outputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
}

void intersectionMany(int const *a, int sizeA, int const *b, int sizeB, int *c, int
*sizeC) {
    int n = 0;
    int i = 0;
    int j = 0;

    //пока индекс массива a не равен размеру массива производим запись
    while (i != sizeA) {
        //если элементы массивов не равны переходим к следующему элементу массива b
        //иначе записываем элемент из массива a, который совпадает с элементом из
массива b, массив c
        if (a[i] != b[j]) {
            j++;
        } else {
            c[n++] = a[i];
            i++;
            j = 0;
        }

        //если дошли до конца массива переходим дальше по массиву a, а индекс массива
b обнуляем
        if (j == sizeB) {
            j = 0;
            i++;
        }
    }

    //записываем размер массива c
    *sizeC = n;
}

int main() {
    int sizeA = 5;
    int sizeB = 5;

    int a[sizeA];
    int b[sizeB];
    int c[sizeA + sizeB];

    inputArray(a, sizeA);
    inputArray(b, sizeB);

    int sizeC = 0;
    intersectionMany(a, sizeA, b, sizeB, c, &sizeC);

    outputArray(c, sizeC);

    return 0;
}
```

3) Разность

```
#include <stdio.h>

void inputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
}

void outputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
}

void differenceSet(int const *a, int sizeA, int const *b, int sizeB, int *c,
int *sizeC) {
    int n = 0;
    int i = 0;
    int j = 0;
    int count = 0;

    //пока индексы не равны размерам массива, ведём запись
    while (i != sizeA) {
        //если элемент из массива a меньше элемента из массива b записываем
        //его, так как массивы отсортированы
        //иначе если элемент из массива a больше элемента из массива b идём
        //дальше по массиву b, счётчик увеличиваем на 1
        //иначе идём дальше по массиву a, так как элементы равны
        if (a[i] < b[j]) {
            c[n++] = a[i];
            i++;
        } else if (a[i] > b[j]){
            j++;
            count++;
        } else {
            i++;
        }

        //если счётчик стал равен размеру массива b(то есть элемента из
        //массива a нет в массиве b)
        //тогда мы записываем элемент a в массив c, идём дальше по массиву a
        //на новый элемент, счётчик и индекс
        //массива b обнуляем
        if (count == sizeB) {
            c[n++] = a[i];
            i++;
            count = 0;
            j = 0;
        }
    }

    //записываем размер массива c
    *sizeC = n;
}
```

```
int main() {  
    int sizeA = 5;  
    int sizeB = 5;  
  
    int a[sizeA];  
    int b[sizeB];  
    int c[sizeA + sizeB];  
  
    inputArray(a, sizeA);  
    inputArray(b, sizeB);  
  
    int sizeC = 0;  
    differenceSet(a, sizeA, b, sizeB, c, &sizeC);  
  
    outputArray(c, sizeC);  
  
    return 0;  
}
```

4) Симметрическая разность

```
#include <stdio.h>

void inputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
}

void outputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
}

void symmetricDifferenceSet(int const *a, int sizeA, int const *b, int sizeB,
int *c, int *sizeC) {
    int n = 0;
    int i = 0;
    int j = 0;

    //пока индексы не равны размерам массива производим запись
    while (i < sizeA && j < sizeB) {
        //если элемент массива a меньше элемента массива b записываем его в
        //массив c и идём дальше по массиву a
        //иначе если элемент массива b меньше элемента массива a записываем
        //его в массив c и идём дальше по массиву b
        //иначе идём дальше по массивам(так как элементы совпадают)
        if (a[i] < b[j]) {
            c[n++] = a[i];
            i++;
        } else if (a[i] > b[j]) {
            c[n++] = b[j];
            j++;
        } else {
            i++;
            j++;
        }
    }

    //пока индекс не равен размеру массива дописываем оставшиеся элементы в
    //конец(если такие есть)
    while (i < sizeA) {
        c[n++] = a[i];
        i++;
    }

    //пока индекс не равен размеру массива дописываем оставшиеся элементы в
    //конец(если такие есть)
    while (j < sizeB) {
        c[n++] = b[j];
        j++;
    }

    //записываем размер массива c
    *sizeC = n;
}
```

```

int main() {
    int sizeA = 6;
    int sizeB = 5;

    int a[sizeA];
    int b[sizeB];
    int c[sizeA + sizeB];

    inputArray(a, sizeA);
    inputArray(b, sizeB);

    int sizeC = 0;
    symmetricDifferenceSet(a, sizeA, b, sizeB, c, &sizeC);

    outputArray(c, sizeC);

    return 0;
}

```

5) Включено ли множество в другое

```

#include <stdio.h>

void inputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
}

int checkingArray(int const * a, int sizeA, int const * b, int sizeB) {
    int i = 0;
    int j = 0;
    int count = 0;

    //пока индекс не равен размеру массива проводим подсчёты
    while (j != sizeB) {
        //если элемент из массива b равен элементу из массива a
        //счётчик увеличиваем на 1 идём дальше по массиву b
        //иначе идём дальше по массиву a
        if (b[j] == a[i]) {
            count++;
            j++;
        } else {
            i++;
        }

        //если индекс i поднялся до максимума, значит элемент из b не был
        //найден
        //тогда мы переходим дальше по массиву b
        if (i == sizeA) {
            j++;
        }
    }

    //если счётчик равен размеру массива b (то есть все элементы из массива b
    //есть в массиве a)
    //возвращаем 1, иначе 0
    return count == sizeB ? 1 : 0;
}

```

```

int main() {
    int sizeA = 6;
    int sizeB = 3;

    int a[sizeA];
    int b[sizeB];

    inputArray(a, sizeA);
    inputArray(b, sizeB);

    if (checkingArray(a, sizeA, b, sizeB)) {
        printf("Yes");
    } else {
        printf("No");
    }

    return 0;
}

```

6) Равны ли массива

```

#include <stdio.h>

void inputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
}

int checkingArray(int const * a, int sizeA, int const * b, int sizeB) {
    //проверяем равны ли длины массива, чтобы не делать лишних проверок
    if (sizeA == sizeB) {
        //определяем совпадают ли элемент элемент из массива a и массива b,
        //если не совпадают к счётчику добавляем 1
        int count = 0;
        int i = 0;
        int j = 0;

        while(i != sizeA) {
            if (a[i] == b[j]) {
                count++;
                i++;
                j++;
            } else {
                i++;
                j++;
            }
        }

        //если все элементы оказались одинаковые возвращаем 1, иначе 0
        return count == sizeA ? 1 : 0;
    } else {
        return 0;
    }
}

```

```
int main() {  
    int sizeA = 5;  
    int sizeB = 5;  
  
    int a[sizeA];  
    int b[sizeB];  
  
    inputArray(a, sizeA);  
    inputArray(b, sizeB);  
  
    if (checkingArray(a, sizeA, b, sizeB)){  
        printf("Yes");  
    } else {  
        printf("No");  
    }  
  
    return 0;  
}
```

7) Строгое включение

```
#include <stdio.h>

void inputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
}

int checkingArray(int const * a, int sizeA, int const * b, int sizeB) {
    int i = 0;
    int j = 0;
    int count = 0;

    //пока индекс не равен размеру массива проводим подсчёты
    while (j != sizeB) {
        //если элемент из массива b равен элементу из массива a
        //счётчик увеличиваем на 1 идём дальше по массиву b
        //иначе идём дальше по массиву a
        if (b[j] == a[i]) {
            count++;
            j++;
        } else {
            i++;
        }

        //если индекс i поднялся до максимума, значит элемент из b не был
        //найден
        //тогда мы переходим дальше по массиву b
        if (i == sizeA) {
            j++;
        }
    }

    //если счётчик равен размеру массива b (то есть все элементы из массива b
    //есть в массиве a)
    //возвращаем 1, иначе 0
    return count == sizeB && sizeA != sizeB ? 1 : 0;
}

int main() {
    int sizeA = 6;
    int sizeB = 6;

    int a[sizeA];
    int b[sizeB];

    inputArray(a, sizeA);
    inputArray(b, sizeB);

    if (checkingArray(a, sizeA, b, sizeB)) {
        printf("Yes");
    } else {
        printf("No");
    }

    return 0;
}
```

8) Дополнение до универсума

```
#include <stdio.h>

void inputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
}

void outputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
}

void additionToUniverse(int *const a, int sizeA, int *const u, int sizeU, int
* const c, int *sizeC) {
    int n = 0;
    int i = 0;
    int j = 0;
    int count = 0;

    //пока индексы не равны размерам массива, ведём запись
    while (i != sizeA) {
        //если элемент из массива a меньше элемента из массива u записываем
        //его, так как массивы отсортированы
        //иначе если элемент из массива a больше элемента из массива u идём
        //дальше по массиву u, счётчик увеличиваем на 1
        //иначе идём дальше по массиву a, так как элементы равны
        if (a[i] < u[j]) {
            c[n++] = a[i];
            i++;
        } else if (a[i] > u[j]){
            j++;
            count++;
        } else {
            i++;
        }

        //если счётчик стал равен размеру массива u(то есть элемента из
        //массива a нет в массиве u)
        //тогда мы записываем элемент a в массив c, идём дальше по массиву a
        //на новый элемент, счётчик и индекс
        //массива b обнуляем
        if (count == sizeU) {
            c[n++] = a[i];
            i++;
            count = 0;
            j = 0;
        }
    }

    //записываем размер массива c
    *sizeC = n;
}

int main() {
    int sizeA = 5;
    int sizeU = 5;

    int a[sizeA];
    int u[sizeU];
```

```
int c[sizeA + sizeU];

inputArray(a, sizeA);
inputArray(u, sizeU);

int sizeC = 0;
additionToUniverse(a, sizeA, u, sizeU, c, &sizeC);

outputArray(c, sizeC);

return 0;

}
```

Задание 3 в)

1) Объединение

```
#include <stdio.h>

void makeBooleanArray(int *a, int size) {
    for (int i = 0; i < size; i++) {
        int number;
        scanf("%d", &number);

        a[number - 1] = 1;
    }
}

void outputBooleanArray(int *a) {
    for (int i = 0; i < 10; i++)
        printf("%d ", a[i]);
    printf("\n");
}

void unionBooleanArray(int *a, int *b, int *c) {
    for (int i = 0; i < 10; i++)
        c[i] = a[i] || b[i];
}

int main() {
    int sizeA = 4;
    int sizeB = 4;

    int a[10] = {0};
    int b[10] = {0};
    int c[10] = {0};

    makeBooleanArray(a, sizeA);
    outputBooleanArray(a);

    makeBooleanArray(b, sizeB);
    outputBooleanArray(b);

    unionBooleanArray(a, b, c);

    outputBooleanArray(c);

    return 0;
}
```

2) Пересечение

```
#include <stdio.h>

void makeBooleanArray(int *a, int size) {
    for (int i = 0; i < size; i++) {
        int number;
        scanf("%d", &number);

        a[number - 1] = 1;
    }
}

void outputBooleanArray(int *a) {
    for (int i = 0; i < 10; i++)
        printf("%d ", a[i]);
    printf("\n");
}

void intersectionBooleanArray(int *a, int *b, int *c) {
    for (int i = 0; i < 10; i++)
        c[i] = a[i] && b[i];
}

int main() {
    int sizeA = 4;
    int sizeB = 4;

    int a[10] = {0};
    int b[10] = {0};
    int c[10] = {0};

    makeBooleanArray(a, sizeA);
    outputBooleanArray(a);

    makeBooleanArray(b, sizeB);
    outputBooleanArray(b);

    intersectionBooleanArray(a, b, c);

    outputBooleanArray(c);

    return 0;
}
```

3) Разность

```
#include <stdio.h>

void makeBooleanArray(int *a, int size) {
    for (int i = 0; i < size; i++) {
        int number;
        scanf("%d", &number);

        a[number - 1] = 1;
    }
}

void outputBooleanArray(int *a) {
    for (int i = 0; i < 10; i++)
        printf("%d ", a[i]);
    printf("\n");
}

void differenceBooleanArray(int *a, int *b, int *c) {
    for (int i = 0; i < 10; i++)
        c[i] = a[i] && !b[i];
}

int main() {
    int sizeA = 5;
    int sizeB = 5;

    int a[10] = {0};
    int b[10] = {0};
    int c[10] = {0};

    makeBooleanArray(a, sizeA);
    outputBooleanArray(a);

    makeBooleanArray(b, sizeB);
    outputBooleanArray(b);

    differenceBooleanArray(a, b, c);

    outputBooleanArray(c);

    return 0;
}
```

4) Симметрическая разность

```
#include <stdio.h>

void makeBooleanArray(int *a, int size) {
    for (int i = 0; i < size; i++) {
        int number;
        scanf("%d", &number);

        a[number - 1] = 1;
    }
}

void outputBooleanArray(int *a) {
    for (int i = 0; i < 10; i++)
        printf("%d ", a[i]);
    printf("\n");
}

void symmetricDifferenceBooleanArray(int *a, int *b, int *c) {
    for (int i = 0; i < 10; i++)
        c[i] = a[i] ^ b[i];
}

int main() {
    int sizeA = 6;
    int sizeB = 5;

    int a[10] = {0};
    int b[10] = {0};
    int c[10] = {0};

    makeBooleanArray(a, sizeA);
    outputBooleanArray(a);

    makeBooleanArray(b, sizeB);
    outputBooleanArray(b);

    symmetricDifferenceBooleanArray(a, b, c);

    outputBooleanArray(c);

    return 0;
}
```

5) Включение

```
#include <stdio.h>
#include <stdbool.h>

void makeBooleanArray(int *a, int size) {
    for (int i = 0; i < size; i++) {
        int number;
        scanf("%d", &number);

        a[number - 1] = 1;
    }
}

void outputBooleanArray(int *a) {
    for (int i = 0; i < 10; i++)
        printf("%d ", a[i]);
    printf("\n");
}

bool inclusionBooleanArray(int *a, int *b) {
    for (int i = 0; i < 10; i++)
        if (a[i] && !b[i]) {
            return false;
        }

    return true;
}

int main() {
    int sizeA = 3;
    int sizeB = 5;

    int a[10] = {0};
    int b[10] = {0};

    makeBooleanArray(a, sizeA);
    outputBooleanArray(a);

    makeBooleanArray(b, sizeB);
    outputBooleanArray(b);

    if (inclusionBooleanArray(a, b)) {
        printf("true");
    } else {
        printf("false");
    }

    return 0;
}
```

6) Строгое включение

```
#include <stdio.h>
#include <stdbool.h>

void makeBooleanArray(int *a, int size) {
    for (int i = 0; i < size; i++) {
        int number;
        scanf("%d", &number);

        a[number - 1] = 1;
    }
}

void outputBooleanArray(int *a) {
    for (int i = 0; i < 10; i++)
        printf("%d ", a[i]);
    printf("\n");
}

bool inclusionBooleanArray(int *a, int *b) {
    for (int i = 0; i < 10; i++)
        if (a[i] && !b[i]) {
            return false;
        }

    return true;
}

int main() {
    int sizeA = 4;
    int sizeB = 5;

    int a[10] = {0};
    int b[10] = {0};

    makeBooleanArray(a, sizeA);
    outputBooleanArray(a);

    makeBooleanArray(b, sizeB);
    outputBooleanArray(b);

    if (sizeA != sizeB && inclusionBooleanArray(a, b)) {
        printf("true");
    } else {
        printf("false");
    }

    return 0;
}
```

7) Равенство

```
#include <stdio.h>
#include <stdbool.h>

void makeBooleanArray(int *a, int size) {
    for (int i = 0; i < size; i++) {
        int number;
        scanf("%d", &number);

        a[number - 1] = 1;
    }
}

void outputBooleanArray(int *a) {
    for (int i = 0; i < 10; i++)
        printf("%d ", a[i]);
    printf("\n");
}

bool equalityBooleanArray(int *a, int *b) {
    for (int i = 0; i < 10; i++)
        if (a[i] != b[i]) {
            return false;
        }

    return true;
}

int main() {
    int sizeA = 5;
    int sizeB = 5;

    int a[10] = {0};
    int b[10] = {0};

    makeBooleanArray(a, sizeA);
    outputBooleanArray(a);

    makeBooleanArray(b, sizeB);
    outputBooleanArray(b);

    if (sizeA == sizeB && equalityBooleanArray(a, b)) {
        printf("true");
    } else {
        printf("false");
    }

    return 0;
}
```

8) Дополнение до универсума

```
#include <stdio.h>

void makeBooleanArray(int *a, int size) {
    for (int i = 0; i < size; i++) {
        int number;
        scanf("%d", &number);

        a[number - 1] = 1;
    }
}

void outputBooleanArray(int *a) {
    for (int i = 0; i < 10; i++)
        printf("%d ", a[i]);
    printf("\n");
}

void additionToUniverseBooleanArray(int *a, int *c) {
    for (int i = 0; i < 10; i++) {
        c[i] = !a[i];
    }
}

int main() {
    int sizeA = 4;

    int a[10] = {0};
    int c[10] = {0};

    makeBooleanArray(a, sizeA);
    outputBooleanArray(a);

    additionToUniverseBooleanArray(a, c);

    outputBooleanArray(c);

    return 0;
}
```

Задание 4 Написать программы для вычисления значений выражений

n1

```
#include <stdio.h>

void inputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
}

void outputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
}

void unionSets(int const *a, int sizeA, int const *b, int sizeB, int *c, int *sizeC) {
    int n = 0;
    //записываем все числа из массива a в массив c
    for (int i = 0; i < sizeA; i++) {
        c[n] = a[i];
        n++;
    }

    //если элемент из массива b не совпадает с элементом из массива a, к счётчику
    добавляем 1
    int count = 0;
    for (int j = 0; j < sizeB; j++) {
        for (int k = 0; k < sizeA; k++) {
            if (b[j] != a[k]) {
                count++;
            }
        }

        //если счётчику стал равен размеру массива a(то есть число из массива b не
        совпало с элементами массива a)
        //записываем этот элемент в массив c
        if (count == sizeA) {
            c[n] = b[j];
            n++;
        }

        count = 0;
    }

    //записываем размер массива c
    *sizeC = n;
}

void intersectionMany(int const * a, int sizeA, int const * b, int sizeB, int *c, int
*sizeC) {
    int n = 0;

    //находим элементы в массиве a и b, которые совпадают
    for (int i = 0; i < sizeA; i++) {
        for (int j = 0; j < sizeB; j++) {
            if (a[i] == b[j]) {
                //записываем эти элементы в массив c
                c[n++] = a[i];
            }
        }
    }

    //сохраняем размер массива c
    *sizeC = n;
}
```

```

void differenceSet(int const *a, int sizeA, int const *b, int sizeB, int *c, int
*sizeC) {
    int n = 0;
    int i = 0;
    int j = 0;
    int count = 0;

    //пока индексы не равны размерам массива, ведём запись
    while (i != sizeA) {
        //если элемент из массива a меньше элемента из массива b записываем его, так
как массивы отсортированы
        //иначе если элемент из массива a больше элемента из массива b идём дальше по
массиву b, счётчик увеличиваем на 1
        //иначе идём дальше по массиву a, так как элементы равны
        if (a[i] < b[j]) {
            c[n++] = a[i];
            i++;
        } else if (a[i] > b[j]){
            j++;
            count++;
        } else {
            i++;
        }
        //если счётчик стал равен размеру массива b (то есть элемента из массива a нет
в массиве b)
        //тогда мы записываем элемент a в массив c, идём дальше по массиву a на новый
элемент, счётчик и индекс
        //массива b обнуляем
        if (count == sizeB) {
            c[n++] = a[i];
            i++;
            count = 0;
            j = 0;
        }
    }
    //записываем размер массива c
    *sizeC = n;
}

int main() {
    int sizeA = 4;
    int sizeB = 4;
    int sizeC = 4;
    int sizeRes1 = 0;
    int sizeRes2 = 0;
    int sizeRes3 = 0;
    int sizeLast1 = 0;

    int a[sizeA];
    int b[sizeB];
    int c[sizeC];
    inputArray(a, sizeA);
    inputArray(b, sizeB);
    inputArray(c, sizeC);

    int res1[sizeA + sizeB];
    int res2[sizeA + sizeC];
    int res3[sizeB + sizeC];
    int lastResult1[sizeRes1 + sizeRes2];

    intersectionMany(a, sizeA, b, sizeB, res1, &sizeRes1);

    differenceSet(a, sizeA, c, sizeC, res2, &sizeRes2);

    differenceSet(b, sizeB, c, sizeC, res3, &sizeRes3);
    outputArray(res3, sizeRes3);

    unionSets(res1, sizeRes1, res2, sizeRes2, lastResult1, &sizeLast1);
    outputArray(lastResult1, sizeLast1);
}

```

Задание 4 п2

```
#include <stdio.h>

void inputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
}

void outputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
}

void unionSets(int const *a, int sizeA, int const *b, int sizeB, int *c, int *sizeC) {
    int n = 0;
    //записываем все числа из массива a в массив c
    for (int i = 0; i < sizeA; i++) {
        c[n] = a[i];
        n++;
    }

    //если элемент из массива b не совпадает с элементом из массива a, к счётчику
    добавляем 1
    int count = 0;
    for (int j = 0; j < sizeB; j++) {
        for (int k = 0; k < sizeA; k++) {
            if (b[j] != a[k]) {
                count++;
            }
        }

        //если счётчику стал равен размеру массива a(то есть число из массива b не
        совпало с элементами массива a)
        //записываем этот элемент в массив c
        if (count == sizeA) {
            c[n] = b[j];
            n++;
        }

        count = 0;
    }

    //записываем размер массива c
    *sizeC = n;
}

void intersectionMany(int const * a, int sizeA, int const * b, int sizeB, int *c, int
*sizeC) {
    int n = 0;

    //находим элементы в массиве a и b, которые совпадают
    for (int i = 0; i < sizeA; i++) {
        for (int j = 0; j < sizeB; j++) {
            if (a[i] == b[j]) {
                //записываем эти элементы в массив c
                c[n++] = a[i];
            }
        }
    }

    //сохраняем размер массива c
    *sizeC = n;
}

int checkingArray(int const * a, int sizeA, int const * b, int sizeB) {
    //проверяем равны ли длины массива, чтобы не делать лишних проверок
    if (sizeA == sizeB) {
        //определяем совпадают ли элемент элемент из массива a и массива b, если не
```

```

совпадают к счётчику добавляем 1
    int count = 0;

    for (int i = 0; i < sizeA; i++) {
        for (int j = 0; j < sizeB; j++) {
            if (a[i] != b[j]) {
                count++;
            }
        }
    }

    //если счётчику равен размеру массива b(то есть был несовпадающий элемент), сразу
    //возвращаем 0
    if (count == sizeB) {
        return 0;
    }

    count = 0;

    //если все элементы оказались одинаковые возвращаем 1
    return 1;
} else {
    return 0;
}
}

int main() {
    int sizeA = 7;
    int sizeB = 5;
    int sizeC = 5;
    int sizeD = 5;
    int sizeRes1 = 0;
    int sizeRes2 = 0;
    int sizeLast = 0;

    int a[sizeA];
    int b[sizeB];
    int c[sizeC];
    int D[sizeD];
    inputArray(a, sizeA);
    inputArray(b, sizeB);
    inputArray(c, sizeC);
    inputArray(D, sizeD);

    int res1[sizeA + sizeB];
    int res2[sizeA + sizeC];
    int lastResult[sizeRes1 + sizeRes2];

    intersectionMany(a, sizeA, c, sizeC, res1, &sizeRes1);
    intersectionMany(a, sizeA, b, sizeB, res2, &sizeRes2);
    unionSets(res2, sizeRes2, res1, sizeRes1, lastResult, &sizeLast);
    if (checkingArray(lastResult, sizeLast, D, sizeD)) {
        printf("(A intersection C) union (A intersection B)");
    } else {
        printf("ERROR");
    }
}

```

Задание 5 Используя программы (см. “Задания”, п.4), вычислить значения выражений

- 1) {5, 6, 3, 1, 8}
- 2) (A intersection C) union (A intersection B)